МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)

Основная образовательная программа 09.03.02
«Информационные системы и технологии»

Отчет по дисциплине «Проектная деятельность»
по домашней работе на семинаре №4

Тема: «Классификация работы спутника по телеметрии»

Проверил
к.т.н. доцент **Ковалев И.А.**

_____
подпись

Выполнила
студентка группы ИДБ-25-06 **Якунина В.С.**

_____
подпись

Москва 2026г

Оглавление

ВВЕДЕНИЕ.....3
ПРОГРАММА.....3
ТЕСТИРОВАНИЕ.....5
ЗАКЛЮЧЕНИЕ.....7

ВВЕДЕНИЕ

Целью работы является изучение классификации работы спутника по телеметрии. Ссылка на github с программой.

ПРОГРАММА

Программа классифицирует данные с помощью разных моделей и сравнивает эти модели.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import (
  accuracy_score,
  precision_score,
  recall_score,
  f1_score,
  classification_report,
  ConfusionMatrixDisplay
)

RANDOM_STATE = 42


def evaluate_binary(y_true, y_pred, title="Модель", average="macro"):
    print("-" * len(title))
    print(title)
    print("-" * len(title))

    print(f"Accuracy : {accuracy_score(y_true, y_pred):.3f}")
```

```
    print(f"Precision: {precision_score(y_true, y_pred, average=average):.3f}")
    print(f"Recall   : {recall_score(y_true, y_pred, average=average):.3f}")
    print(f"F1-score : {f1_score(y_true, y_pred, average=average):.3f}")

    print("\nПодробный отчет:")
    print(classification_report(y_true, y_pred, digits=3))

    ConfusionMatrixDisplay.from_predictions(
      y_true,
      y_pred,
      cmap="Blues"
    )
    plt.title(title)
    plt.show()

def main():
  filename = "satellite_telemetry_modes.csv"
  df = pd.read_csv(filename)
  print("Размер датасета:", df.shape)
  print()
  print(df.head())
  print()
  print(df.info())
  print()

  print("\nПропущенные значения:")
  print(df.isna().sum())
  print()

  print("\nКоличество объектов каждого класса:")
  print(df["mode"].value_counts())
  print()

  plt.figure(figsize=(8, 6))

  for class_value, label in [("IDLE", "Класс IDLE"), ("OBSERVATION", "Класс
OBSERVATION"), ("TRANSMISSION", "Класс TRANSMISSION")]:
    part = df[df["mode"] == class_value]
    plt.scatter(
      part["cpu_load"],
      part["memory_load"],
      label=label,
      alpha=0.5,
      s=25
    )

  plt.xlabel("CPU load")
  plt.ylabel("Memory load")
  plt.title("Telemetry: распределение классов")
```

```
  plt.legend()
  plt.grid(alpha=0.2)

  plt.figure(figsize=(8, 6))

  for class_value, label in [("IDLE", "Класс IDLE"), ("OBSERVATION", "Класс
OBSERVATION"), ("TRANSMISSION", "Класс TRANSMISSION")]:
    part = df[df["mode"] == class_value]
    plt.scatter(
      part["communication_load"],
      part["power_load"],
      label=label,
      alpha=0.5,
      s=25
    )

  plt.xlabel("Communication load")
  plt.ylabel("Power load")
  plt.title("Telemetry: распределение классов")
  plt.legend()
  plt.grid(alpha=0.2)

  plt.show()

  X = df[["cpu_load", "memory_load", "communication_load", "power_load"]]
  y = df["mode"]

  X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y
  )

  print()
  print("Обучающая выборка:", X_train.shape)
  print("Тестовая выборка:", X_test.shape)
  print()

  scaler = StandardScaler()

  X_train_scaled = scaler.fit_transform(X_train)
  X_test_scaled = scaler.transform(X_test)

  # k-NN
  knn = KNeighborsClassifier(n_neighbors=5)
  knn.fit(X_train_scaled, y_train)
  y_pred_knn = knn.predict(X_test_scaled)
```

```
evaluate_binary(y_test, y_pred_knn, "k-NN, k = 5")

# Logistic Regression
log_reg = LogisticRegression(
  max_iter=1000,
  random_state=RANDOM_STATE
)
log_reg.fit(X_train_scaled, y_train)
y_pred_log = log_reg.predict(X_test_scaled)
evaluate_binary(y_test, y_pred_log, "Логистическая регрессия")
probabilities = log_reg.predict_proba(X_test_scaled[:10])
result = pd.DataFrame(
  probabilities,
  columns=["P(class=IDLE)", "P(class=OBSERVATION)", "P(class=TRANSMISSION)"]
)
result["prediction"] = log_reg.predict(X_test_scaled[:10])
result["real_class"] = y_test.iloc[:10].values
result

#  Decision Tree
tree = DecisionTreeClassifier(
  max_depth=4,
  random_state=RANDOM_STATE
)
tree.fit(X_train, y_train)
y_pred_tree = tree.predict(X_test)
evaluate_binary(y_test, y_pred_tree, "Дерево решений, max_depth = 4")
feature_importance_tree = pd.Series(
  tree.feature_importances_,
  index=X.columns
).sort_values(ascending=False)
feature_importance_tree

#  Random Forest
random_forest = RandomForestClassifier(
  n_estimators=200,
  max_depth=6,
  random_state=RANDOM_STATE,
  n_jobs=-1
)
random_forest.fit(X_train, y_train)
y_pred_rf = random_forest.predict(X_test)
evaluate_binary(y_test, y_pred_rf, "Random Forest")
feature_importance_rf = pd.Series(
  random_forest.feature_importances_,
  index=X.columns
).sort_values(ascending=False)
feature_importance_rf
```

```
  # Naive Bayes
  gaussian_nb = GaussianNB()
  gaussian_nb.fit(X_train, y_train)
  y_pred_gnb = gaussian_nb.predict(X_test)
  evaluate_binary(y_test, y_pred_gnb, "Gaussian Naive Bayes")

  # Общее сравнение
  som_predictions = {
    "k-NN": y_pred_knn,
    "Logistic Regression": y_pred_log,
    "Decision Tree": y_pred_tree,
    "Random Forest": y_pred_rf,
    "Gaussian Naive Bayes": y_pred_gnb
  }

  rows = []

  for model_name, prediction in som_predictions.items():
    rows.append({
      "Model": model_name,
      "Accuracy": accuracy_score(y_test, prediction),
      "Precision": precision_score(y_test, prediction, average="macro"),
      "Recall": recall_score(y_test, prediction, average="macro"),
      "F1": f1_score(y_test, prediction, average="macro")
    })

  som_results = pd.DataFrame(rows).sort_values(
    "F1",
    ascending=False
  )

  som_results.round(3)

  arr = ["Model", "Accuracy", "Precision", "Recall", "F1"]
  print("|        Model        |      Accuracy       |      Precision      |
Recall        |         F1          |")

print("|---------------------|---------------------|---------------------|---------
------------|---------------------|")
  for i in rows:
    print("|", end='')
    for j in arr:
      print(f"{i[j]:21}", end='|')
    print('\n', end='')




if __name__ == "__main__":
```

```
main()
```

ТЕСТИРОВАНИЕ

Была проведена проверка работы программы. Результаты проверки представлены на рисунках 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 и 15.

```
Размер датасета: (600, 6)

   cpu_load  memory_load  power_load  communication_load          mode  mode_num
0        26           35          34                  39          IDLE         0
1        65           59          53                  75  TRANSMISSION         2
2        24           46          63                  76  TRANSMISSION         2
3        34           49          42                  29          IDLE         0
4        19           47          33                  32          IDLE         0

<class 'pandas.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   cpu_load           600 non-null    int64
 1   memory_load        600 non-null    int64
 2   power_load         600 non-null    int64
 3   communication_load  600 non-null    int64
 4   mode               600 non-null    str
 5   mode_num           600 non-null    int64
dtypes: int64(5), str(1)
memory usage: 28.3 KB
None


Пропущенные значения:
cpu_load             0
memory_load          0
power_load           0
communication_load   0
mode                 0
mode_num             0
dtype: int64
```

Рисунок 1.

```
Количество объектов каждого класса:
mode
IDLE            200
TRANSMISSION    200
OBSERVATION     200
Name: count, dtype: int64
```

Рисунок 2.

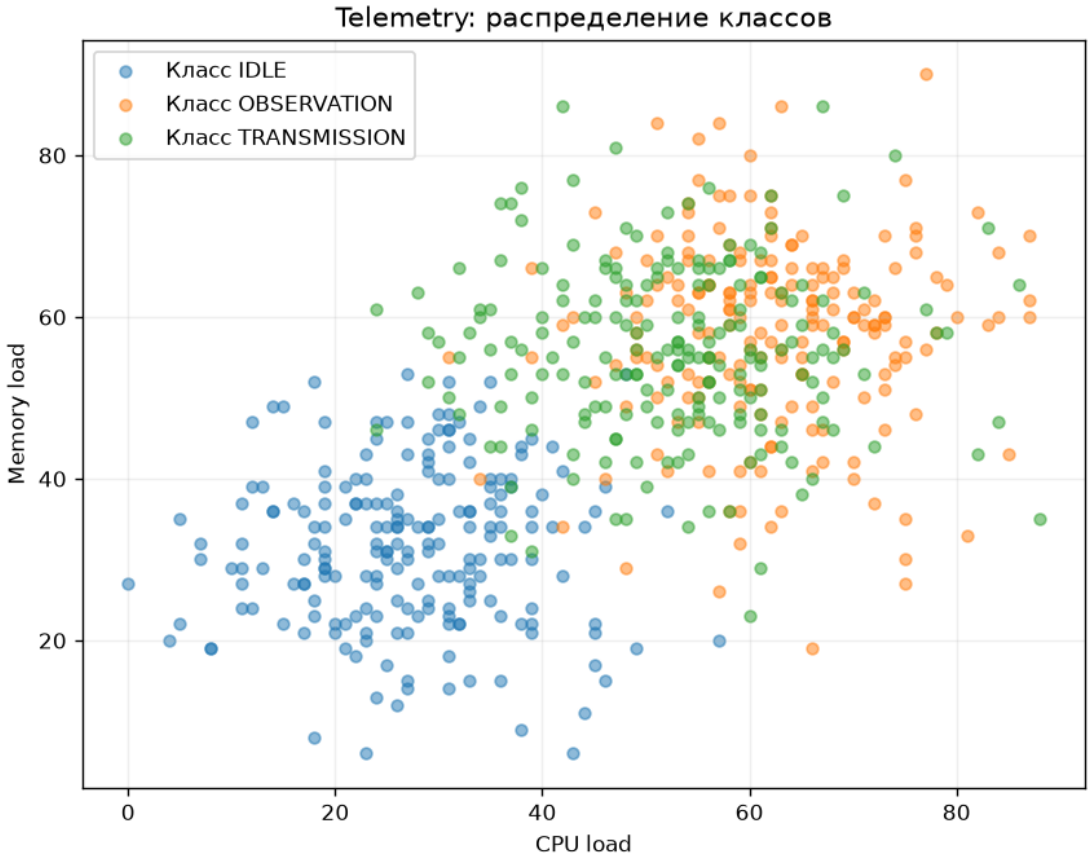


Рисунок 3.

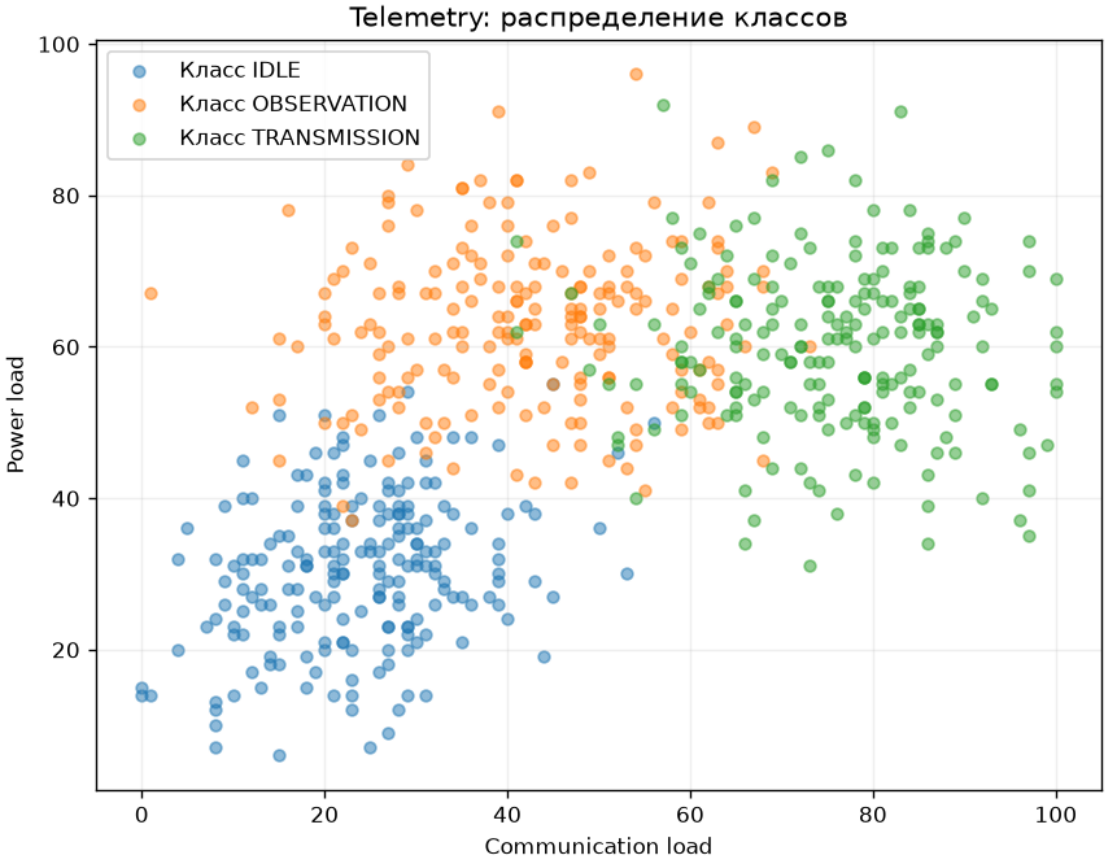

Telemetry: распределение классов
Класс IDLE
Класс OBSERVATION
Класс TRANSMISSION
Power load
Communication load
100
80
60
40
20
0
20
40
60
80
100

Рисунок 4.

```
Обучающая выборка: (480, 4)
Тестовая выборка: (120, 4)

-----------
k-NN, k = 5
-----------
Accuracy : 0.942
Precision: 0.943
Recall   : 0.942
F1-score : 0.942

Подробный отчет:
              precision    recall  f1-score   support

        IDLE      1.000     1.000     1.000        40
 OBSERVATION      0.946     0.875     0.909        40
TRANSMISSION      0.884     0.950     0.916        40

    accuracy                          0.942       120
   macro avg      0.943     0.942     0.942       120
weighted avg      0.943     0.942     0.942       120
```

Рисунок 5.

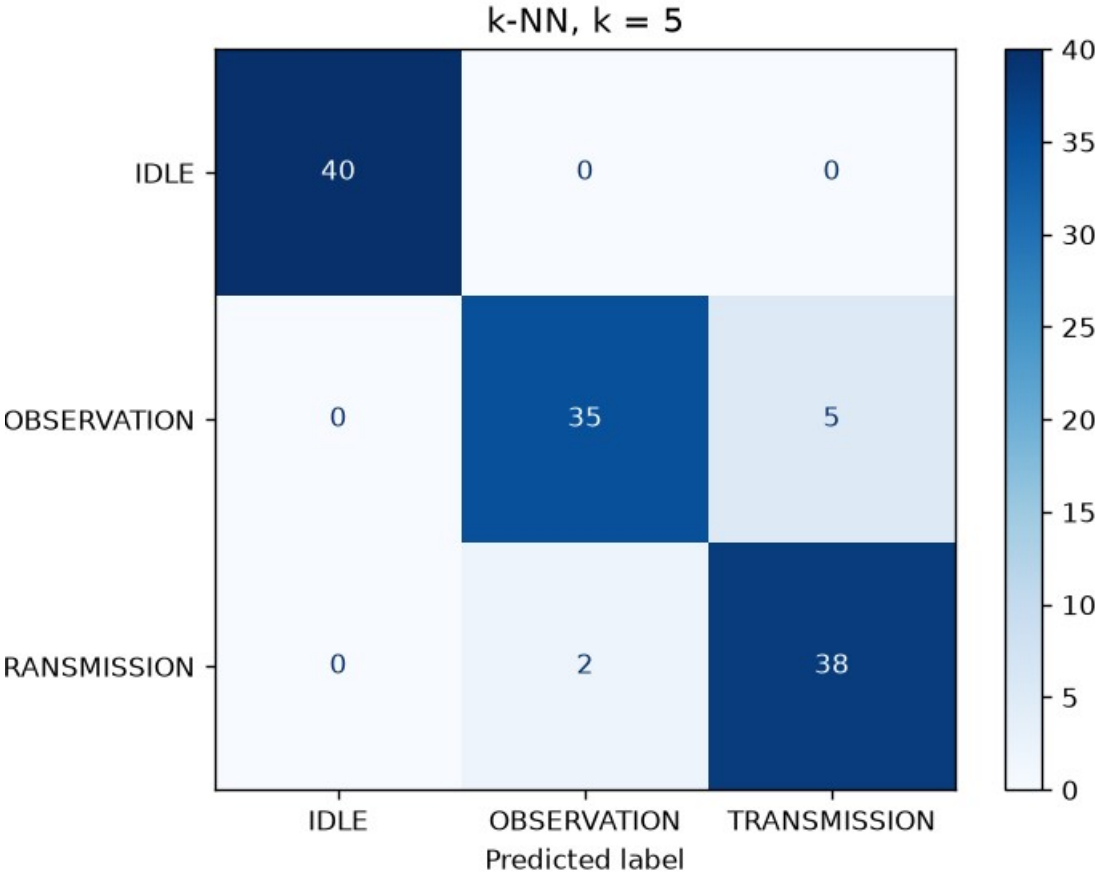


Рисунок 6.

```
----------------------
Логистическая регрессия
----------------------
Accuracy : 0.917
Precision: 0.918
Recall   : 0.917
F1-score : 0.917

Подробный отчет:
              precision    recall  f1-score   support

        IDLE       0.975     0.975     0.975        40
 OBSERVATION       0.919     0.850     0.883        40
TRANSMISSION       0.860     0.925     0.892        40

    accuracy                          0.917       120
   macro avg       0.918     0.917     0.917       120
weighted avg       0.918     0.917     0.917       120
```

Рисунок 7.

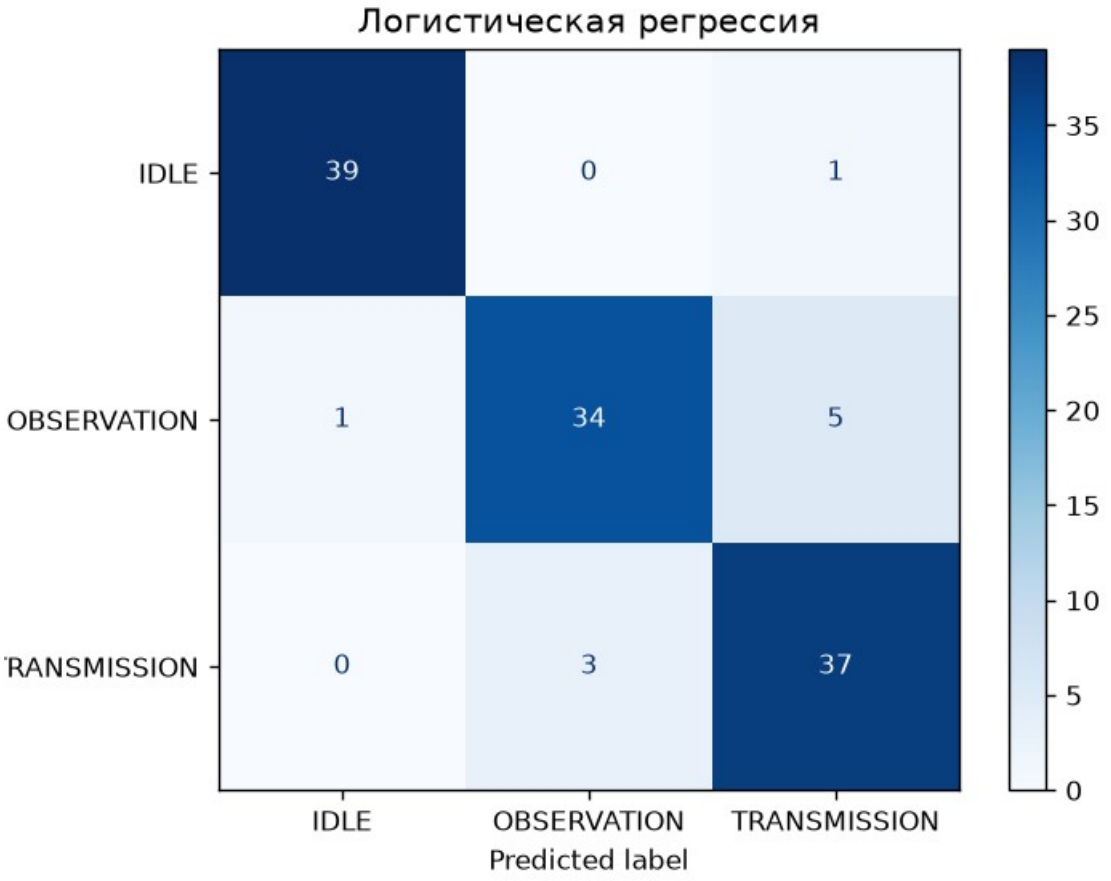


Рисунок 8.

```
----------------------------
Дерево решений, max_depth = 4
----------------------------
Accuracy : 0.908
Precision: 0.907
Recall   : 0.908
F1-score : 0.908

Подробный отчет:
              precision    recall  f1-score   support

        IDLE      0.976     1.000     0.988        40
 OBSERVATION      0.875     0.875     0.875        40
TRANSMISSION      0.872     0.850     0.861        40

    accuracy                          0.908       120
   macro avg      0.907     0.908     0.908       120
weighted avg      0.907     0.908     0.908       120
```

Рисунок 9.

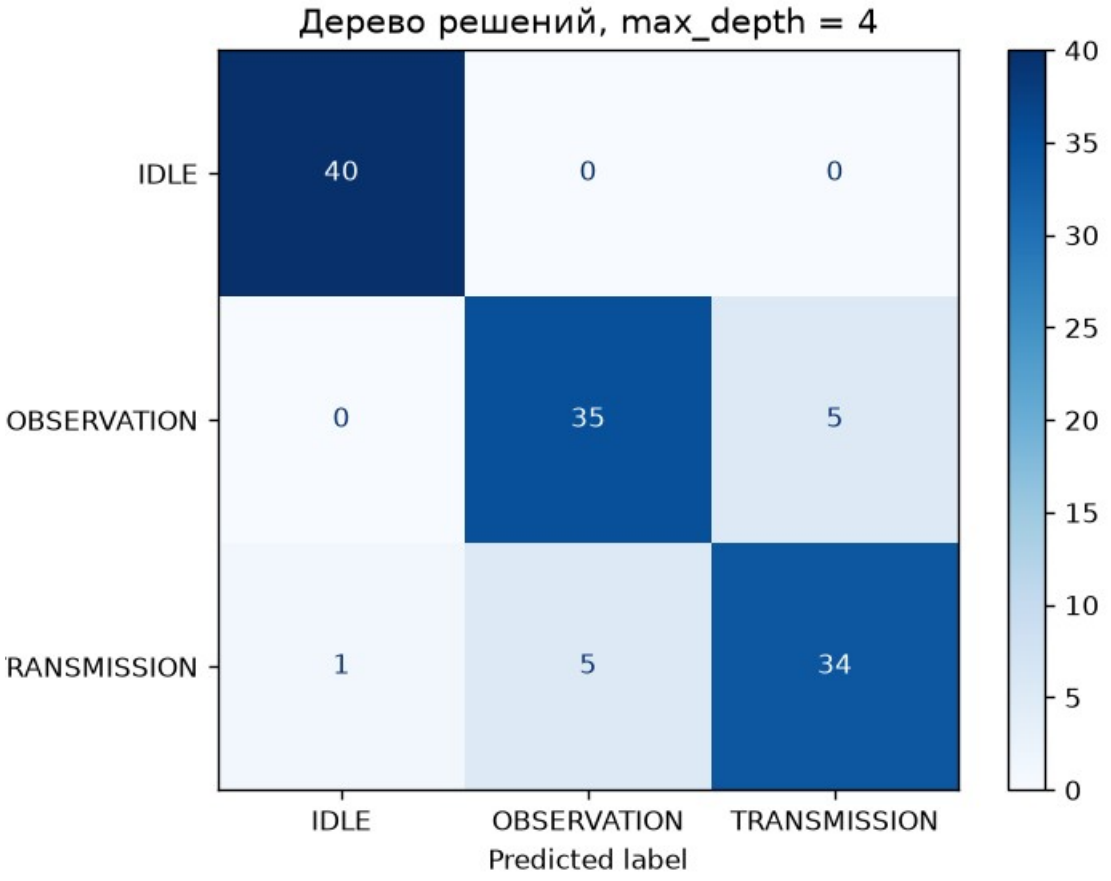


Рисунок 10.

```
------------
Random Forest
------------
Accuracy : 0.950
Precision: 0.950
Recall   : 0.950
F1-score : 0.950

Подробный отчет:
              precision    recall  f1-score   support

        IDLE       0.976     1.000     0.988        40
 OBSERVATION       0.947     0.900     0.923        40
TRANSMISSION       0.927     0.950     0.938        40

    accuracy                           0.950       120
   macro avg       0.950     0.950     0.950       120
weighted avg       0.950     0.950     0.950       120
```

Рисунок 11.

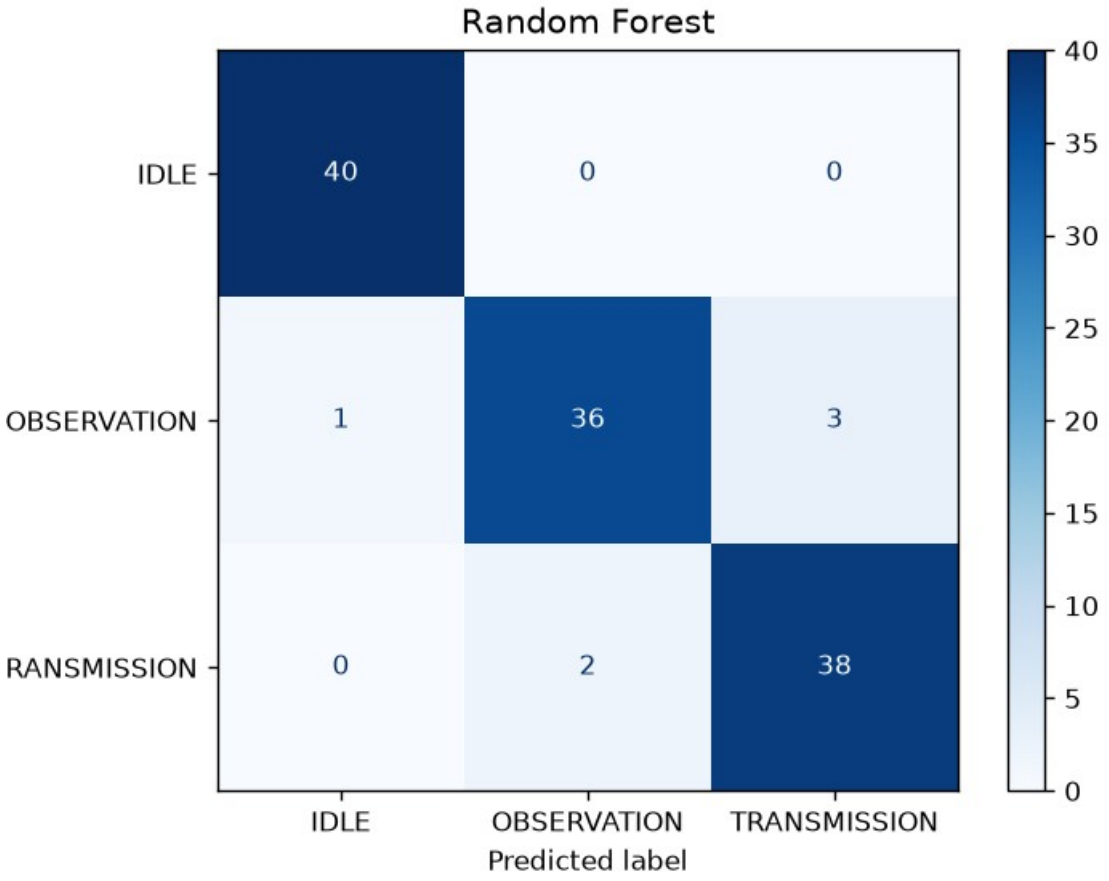


Рисунок 12.

```
--------------------
Gaussian Naive Bayes
--------------------
Accuracy : 0.925
Precision: 0.930
Recall   : 0.925
F1-score : 0.925

Подробный отчет:
              precision    recall  f1-score   support

        IDLE       1.000     0.975     0.987        40
 OBSERVATION       0.944     0.850     0.895        40
TRANSMISSION       0.844     0.950     0.894        40

    accuracy                          0.925       120
   macro avg       0.930     0.925     0.925       120
weighted avg       0.930     0.925     0.925       120
```

Рисунок 13.

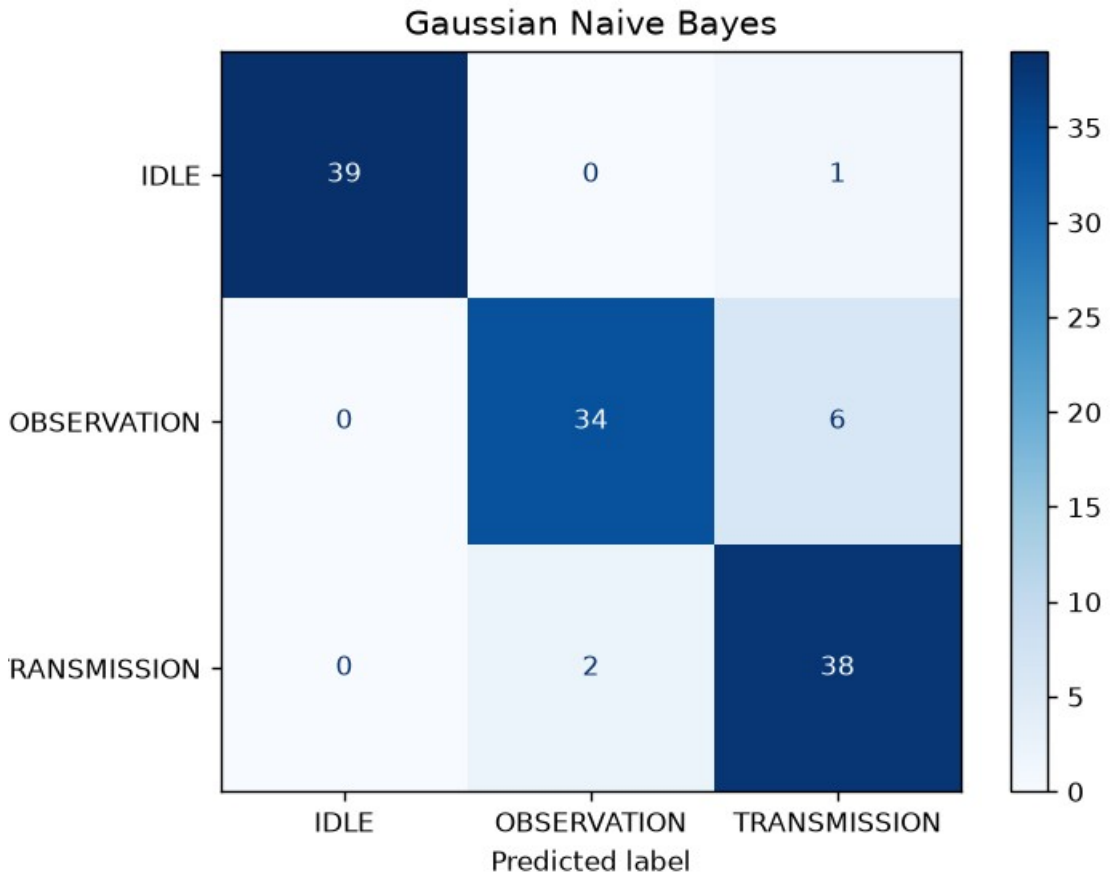


Рисунок 14.

Model	Accuracy	Precision	Recall	F1
k-NN	0.9416666666666667	0.9432222920595015	0.9416666666666668	0.9415845198977729
Logistic Regression	0.9166666666666666	0.9181280117326628	0.9166666666666666	0.9165610493923747
Decision Tree	0.9083333333333333	0.9074682092974776	0.9083333333333333	0.9078046048861802
Random Forest	0.95	0.949935815147625	0.9499999999999998	0.949667616334283
Gaussian Naive Bayes	0.925	0.9296296296296296	0.9249999999999999	0.9253987537719951

Рисунок 15.

ЗАКЛЮЧЕНИЕ

В результате выполнения задания были изучены метрики и их способы вычисления, а также прошло сравнение с расчетами компьютера. А также был выбран алгоритм, больше всего подходящий для классификации режима спутника — k-NN. K-NN, потому что эта модель выдавала точные значения метрик и допустила чуть меньше ошибок, чем остальные. Ссылка на github с программой.